

Estructura de Computadores

Memoria

Laboratorio 1

Daniel Deblas Payo / 100522144 / 100522144@alumnos.uc3m.es

Xiya Ye / 100522159 / 100522159@alumnos.uc3m.es

Grado de Ingeniería Informática - Grupo 85

27 de octubre del 2024

Índice

Índice.....	2
Ejercicio 1.....	3
● Función f.....	3
Explicación funcionamiento de la función.....	3
Pseudocódigo.....	3
Batería de pruebas.....	4
● Función compute_integral.....	5
Explicación funcionamiento de la función.....	5
Pseudocódigo.....	5
Batería de pruebas.....	6
Ejercicio 2.....	7
● Función Integral_Matrix.....	7
Explicación funcionamiento de la función.....	7
Pseudocódigo.....	8
Batería de pruebas.....	8
Conclusión.....	9

Ejercicio 1

- **Función f**

Explicación funcionamiento de la función

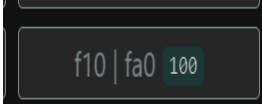
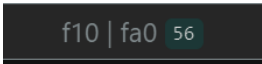
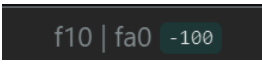
La función f tiene como objetivo calcular el valor de $f(x) = p \cdot x^2 + q \cdot x + r$. Para ello, la función recibe 4 argumentos (p,q,r,x) en los registros (a0, a1, a2, a3, fa0) respectivamente.

Primero se apilan los registros fs2...fs5 además de los registros que guardan los datos que recibe la función. Para calcular x^2 se usa la función pow que guarda el resultado en fa0. Posteriormente se calculan los valores de $p \cdot x^2$, $q \cdot x$ y r y se guardan en los registros f2, fs3, fs4 respectivamente. Después se suman estos registros para dar el resultado final en el registro fs5. Por último se mueve el valor de fs5 al registro fa0 cumpliendo así con el convenio de paso de parámetros.

Pseudocódigo

```
def f(p:int, q:int, r:int, x:float) -> float:
    #X^2 -> Función pow
    x_cuadrado = x**2
    #Resultado final
    resultado = p*x_cuadrado + q*x + r
    return resultado
```

Batería de pruebas

<u>Datos a introducir</u>	<u>Descripción de la prueba</u>	<u>Resultado esperado</u>	<u>Resultado obtenido</u>
$x = 5$ $a0 = p = 3$ $a1 = q = 4$ $a2 = r = 5$	Todos los valores > 0 Se comprueba un correcto funcionamiento con valores pequeños.	100	
$x = 54$ $a0 = p = 35$ $a1 = q = 49$ $a2 = r = 56$	Todos los valores > 0 Se comprueba un correcto funcionamiento con valores mayores.	104762	
$x = 0$ $a0 = p = 35$ $a1 = q = 49$ $a2 = r = 56$	$x = 0$ Se comprueba un correcto funcionamiento cuando $x = 0$	56	
$x = 0$ $a0 = p = 35$ $a1 = q = 49$ $a2 = r = -56$	$x = 0$ y $r < 0$ Se comprueba un correcto funcionamiento cuando $x = 0$ y uno de los valores es < 0.	-56	
$x = -10$ $a0 = p = 3$ $a1 = q = 40$ $a2 = r = 0$	$x < 0$ Se comprueba un correcto funcionamiento con un valor negativo de x	-100	

- **Función compute_integral**

Explicación funcionamiento de la función

La función compute_integral tiene como objetivo calcular la integral definida de una función cuadrática. Para ello va a recibir diferentes parámetros y va a usar la aproximación:

$$\int_a^b f(x)dx \sim h \sum_{n=0}^{N-1} f(a + n \cdot h)$$

Para conseguir esto, la función va a recibir 6 parámetros entre los registros a0 y a5. Estos parámetros son dos números enteros (a,b) que son los límites de la integral, tres números enteros (p, q, r) que se corresponden con los valores de la función y por último un valor N que se usará en los cálculos de la función.

Esta función va a calcular un valor $h = (b-a)/N$ que va a servir para posteriormente calcular el valor de x ($x = a+n \cdot h$) donde el valor de n se va a ir actualizando con un bucle (0-1000).

Con el bucle, la función va a calcular el valor de x con todos los valores de n . Después moverá los valores p, q y r a los registros correspondientes para posteriormente llamar a la función f y de esta forma calcular el valor de la función.

Después de este cálculo, actualizamos el valor de n y el valor de la suma con el resultado obtenido.

Por último, multiplicamos h por el valor de la suma total y lo movemos al registro fa0 para devolverlo.

Pseudocódigo

```
def compute_integral(a:int, b:int, p:int, q:int, r:int, N = 1000) -> float:
    #Iniciamos el valor de la suma
    suma = 0.0
    #Calculamos h
    h = (b-a)/N
    #Con un bucle, calculamos el valor de todos los subintervalos y los guardamos en la variable suma
    n = 1
    while n < N:
        suma += f(p,q,r,(a+n*h))
        n+= 1
    #El resultado es el valor de la suma*h
    resultado = suma*h
    return resultado
```

Batería de pruebas

<u>Datos a introducir</u>	<u>Descripción de la prueba</u>	<u>Resultado esperado</u>	<u>Resultado obtenido</u>
a = 2 b = 5 p = 2 q = 7 r = 2	Se comprueba un correcto funcionamiento con todos los valores > 0.	157.405509	f10 fa0 157.4056...
a = 5 b = 2 p = 2 q = 7 r = 2	Se comprueban los mismos valores de la prueba anterior intercambiando a y b.	-157.59450900	f10 fa0 -157.594...
a = 0 b = 0 p = 2 q = 7 r = 2	Se comprueba el resultado cuando ambos valores (a y b) son 0.	0.0	f10 fa0 0
a = 3 b = -5 p = 2 q = 7 r = 2	Se comprueba el funcionamiento con un valor negativo para b	-29.191	f10 fa0 -29.1912...
a = -3 b = -6 p = 2 q = 7 r = 2	Se comprueba el funcionamiento con valores < 0 para a y b	-37.4505	f10 fa0 -37.4505...

*Se ha usado un valor N = 1000 para todas las pruebas.

Ejercicio 2

- **Función Integral_Matrix**

Explicación funcionamiento de la función

La función Integral_Matrix crea una matriz de dimensión MxM con los siguientes valores:

$$\begin{aligned} \text{matriz}[a][b] &= \int_a^b f(x)dx && \text{Si } b > a \\ \text{matriz}[a][b] &= 0 && \text{Si } b \leq a \end{aligned}$$

Esta función recibe la dirección inicial de la matriz, M, p, q, r.

Para el desarrollo de la matriz, lo primero que hacemos es guardar los registros s que se usan en la función y el registro ra en la pila para seguir el convenio de paso de parámetros. Posteriormente, movemos los datos que recibe la función en registros s, y después inicializamos la variable a a 0.

Debido a que tenemos que hacer una matriz, hemos hecho dos bucles anidados, para hacer las filas y columnas. El primer bucle va a tener como condición que termine cuando a sea mayor o igual que M, porque la matriz debe ser de dimensión MxM. Una vez hecho eso, inicializamos el valor de b a 0 para ir añadiendo los valores de las filas.

Para eso, creamos otro bucle dentro, con la misma condición que el anterior, sólo que este con b. Una vez dentro del bucle, calculamos la dirección: $\text{dir_inicial} + (a * M + b) * P$, siendo $P = 4$, el tamaño de lo que estamos moviendo.

Una vez calculado la dirección, según el valor de a y b, o calculamos la integral, llamando a la función compute_integral desarrollada en el ejercicio 1, que nos guarda el resultado en fa0, entonces lo guardamos en la memoria, siendo la dirección que hemos calculado previamente.

En caso de que suceda $b \leq a$, guardamos un 0 en a0 y hacemos lo mismo que con fa0, lo guardamos en la memoria, con la dirección calculada. Después, vamos iterando los valores de a y b.

Una vez terminado la matriz, desapilamos los registros s y el registro ra, cumpliendo así con el convenio de paso de parámetros.

Pseudocódigo

```
def integral_matrix(m:int, p:int, q:int, r:int, n:int) -> list:
    matriz = [] # creamos la lista para guardar la matriz
    a = 0 # inicializamos a a 0
    while a < m:
        b = 0 # inicializamos b a 0
        fila = [] # creamos la lista para las filas
        while b < m:
            if b > a: # si b < a, calculamos la integral
                # calculamos la integral:
                result = self.compute_integral(a, b, p, q, r, n)
                fila.append(result) # añadimos el resultado en la matriz
            else: # si b >= a, añadimos un 0
                result = 0
                fila.append(result) # añadimos un 0 a la matriz
            b += 1
        matriz.append(fila) # añadimos la fila a la matriz
        a += 1
    return matriz
```

Batería de pruebas

<u>Datos a introducir</u>	<u>Descripción de la prueba</u>	<u>Resultado esperado</u>	<u>Resultado obtenido</u>
M = 5 p = 2 q = 3 r = 4	Se comprueba el funcionamiento para una matriz 5x5 y todos los valores >0 .	0.0 / 6.14 / 19.19 / 43.1 / 81.79 / 0.0 / 0.0 / 13.12 / 37.11 / 75.92 / 0.0 / 0.0 / 0.0 / 24.1 / 63.03 / 0.0 / 0.0 / 0.0 / 0.0 / 39.08 / 0.0 / 0.0 / 0.0 / 0.0 / 0.0 /	0 6,14 19,19 43,10 81,79 0 0 13,13 37,13 75,95 0 0 0 24,11 63,08 0 0 0 0 39,10 0 0 0 0 0
M = 3 p = -1 q = 2 r = -5	Se comprueba que para una matriz 3x3, con valores de la función <0 funciona.	0.0 / -4.34 / -8.67 0.0 / 0.0 / -4.33 / 0.0 / 0.0 / 0.0 /	0 -4,34 -8,67 0 0 -4,33 0 0 0
M = 4 p = 6412 q = 85650 r = 67452	Se comprueba el funcionamiento para una matriz 4x4, con valores muy grandes.	0.0 / 111954.13 / 321334.04 / 640772.02 0.0 / 0.0 / 210364.01 / 530849.56 / 0.0 / 0.0 / 0.0 / 321597.89 / 0.0 / 0.0 / 0.0 / 0.0 /	0 111.954,12 321.334,06 640.772,06 0 0 210403,50 531008,00 0 0 0 321.687,38 0 0 0 0
M = 3 p = -8512 q = -5563 r = -7412	Se comprueba el funcionamiento para una matriz 3x3, con valores muy pequeños.	0.0 / -12960.6 / -48198.06 0.0 / 0.0 / -35462.48 / 0.0 / 0.0 / 0.0 /	0 -12960,60 -48198,06 0 0 -35469,34 0 0 0

M = 10 p = 1 q = -2 r = -3	Se comprueba el funcionamiento para una matriz de 10x10.	<pre> 0.0 / -3.66 / -7.33 / -9.04 / -6.83 / 1.29 / 17.28 / 43.11 / 80.76 / 132.18 0.0 / 0.0 / -3.67 / -5.37 / -3.13 / 5.01 / 21.04 / 46.92 / 84.62 / 136.12 / 0.0 / 0.0 / 0.0 / -1.68 / 0.99 / 8.78 / 24.85 / 50.79 / 86.58 / 140.13 / 0.0 / 0.0 / 0.0 / 0.0 / 2.31 / 10.60 / 26.82 / 52.89 / 89.54 / 142.12 / 0.0 / 0.0 / 0.0 / 0.0 / 0.0 / 5.3 / 26.51 / 59.6 / 88.65 / 140.29 / 0.0 / 0.0 / 0.0 / 0.0 / 0.0 / 0.0 / 16.29 / 42.47 / 89.51 / 132.37 / 0.0 / 0.0 / 0.0 / 0.0 / 0.0 / 0.0 / 26.28 / 64.43 / 116.42 / 0.0 / 0.0 / 0.0 / 0.0 / 0.0 / 0.0 / 38.27 / 90.39 / 0.0 / 0.0 / 0.0 / 0.0 / 0.0 / 0.0 / 52.26 / 0.0 / 0.0 / 0.0 / 0.0 / 0.0 / 0.0 / 0.0 / 0.0 / </pre> <pre> 0 3.66 -7.33 -9.04 -6.83 1.29 17.28 43.11 80.76 132.18 0 0 -3.67 -5.37 -3.13 5.02 21.06 46.95 84.66 136.17 0 0 0 -1.68 0.61 8.82 24.94 50.94 88.79 140.47 0 0 0 0 2.32 10.60 26.82 52.94 90.96 142.83 0 0 0 0 0 5.32 26.62 50.85 89.02 141.09 0 0 0 0 0 0 16.33 42.65 80.93 133.17 0 0 0 0 0 0 0 26.34 64.70 117.05 0 0 0 0 0 0 0 0 38.36 90.76 0 0 0 0 0 0 0 0 0 52.38 0 0 0 0 0 0 0 0 0 0 </pre>
-------------------------------------	--	--

Conclusión

La realización de esta práctica nos ha ayudado a afianzar los conocimientos sobre la programación en ensamblador. Al realizarla, hemos entendido cómo funciona la pila y como se debe usar el convenio de paso de parámetros.

La principal dificultad que hemos encontrado a la hora de hacer el programa es usar correctamente la pila y los registros para que se guarden correctamente los valores y que no se vieran afectados al usar la función pow.

En total, calculamos que aproximadamente le hemos dedicado entre 20 y 25 horas a la realización de la práctica.